

Report Indizes

Daten

Ich habe mit Python und Faker (de_AT) 500.000 Personen in die Tabelle personen geschrieben. Weil ich Faker.seed(42) verwende, kommen bei jedem Durchlauf die gleichen Namen raus.

Streuung

Die Varianz kann man bei Namen nicht einfach berechnen, weil es zwischen zwei Namen keinen Abstand gibt. Deshalb habe ich gezählt, wie oft jeder Vorname vorkommt, und das angeschaut.

```
SELECT COUNT(DISTINCT vorname)
FROM personen;
```

```
SELECT MIN(anzahl), MAX(anzahl), AVG(anzahl)
FROM (
  SELECT COUNT(*) AS anzahl
  FROM personen
  GROUP BY vorname
);
```

Es gibt 559 verschiedene Vornamen. Wenn alles gleich verteilt wäre, müsste jeder Name $500.000 / 559 = \text{ca. } 894$ mal vorkommen. Der seltenste Name kommt 810 mal vor, das ist nicht weit weg. Der häufigste Name ist Nikola mit 1786, also ca. doppelt so oft. Im Quellcode von Faker (providers/person/de_AT) sieht man, dass Nikola bei den Männernamen und bei den Frauennamen steht. Faker nimmt beide Listen zusammen und zieht daraus zufällig einen Namen, deshalb hat Nikola doppelt so viele Chancen. Die Listen haben zusammen 560 Einträge (239 + 321), in meiner Datenbank sind aber nur 559 verschiedene Namen, weil Nikola nur einmal gezählt wird. Sonst verteilt Faker die Namen ziemlich gleichmäßig.

Index

```
CREATE INDEX idx_vorname ON personen(vorname);
```

	ohne Index	mit Index
Suche nach Vorname	83 ms	9 ms
Größe DB	11,0 MB	18,2 MB

Mit Index ist die Suche ca. 9 mal schneller. Ohne Index muss SQLite jede Zeile durchgehen, mit Index sind die Vornamen sortiert gespeichert und es findet sie direkt. Dafür ist die Datenbank um ca. 7 MB größer geworden, weil der Index extra gespeichert wird.

Datenbank mit Bias

Hier heißen 50% der Personen Anna.

Suche nach	ohne Index	mit Index
Anna (250.095 Treffer)	88 ms	135 ms
Adam (472 Treffer)	64 ms	2 ms

Bei Adam hilft der Index genauso wie vorher. Bei Anna wird es mit Index sogar langsamer. Das liegt daran, dass die Hälfte der Tabelle gefunden wird. SQLite muss dann für jeden Treffer vom Index wieder in die Tabelle springen, und das dauert länger als einfach alles durchzulesen.

Fazit

Ein Index bringt viel, wenn man nach etwas sucht, das selten vorkommt. Wenn ein Wert sehr oft vorkommt, bringt er nichts oder macht es sogar langsamer. Außerdem braucht er zusätzlichen Speicher.